Method and Baseline

NLP-Based Resume-Job Alignment | Step 2, Week 2

Tejaswi Chava | Prepared November 3, 2026. Preliminary computational evidence; no final research conclusion.

1. Research question and implemented method

Does combining semantic similarity with lexical matching improve identification of relevant resume-job qualifications compared with lexical matching alone? The working hypothesis is that semantic information may help with differently worded qualifications. The Week 2 experiment records initial measurements for the comparison proposed in Week 1.

The lexical baseline uses lowercase unigram TF-IDF with default tokenization and no stop-word removal. It fits the vocabulary and IDF weights on the fixed resume candidate collection only, transforms each job, and calculates cosine similarity. This follows the Week 1 method; scores are recomputed on the expanded collection, so they are not directly comparable with Week 1 scores.

The semantic method uses pretrained all-MiniLM-L6-v2 embeddings through the Xenova full-precision ONNX export. Tokenization is truncated at 256 tokens, followed by attention-mask mean pooling of token embeddings and L2 normalization. Dot products of normalized vectors give cosine similarity. The hybrid is 0.5 x lexical cosine + 0.5 x semantic cosine. Raw scores are not calibrated probabilities. No model training, fine-tuning, weight search, or label-based scoring is performed.

2. Source notebook and responsible sections

The Week 2 implementation follows the original plan in src/week1_resume_job_foundation.ipynb. Its six sections verify the frozen benchmark, obtain the pretrained model, define scoring and metrics, test the calculations, run the comparison, and inspect candidate rankings. Recorded observations follow the implementation. src/requirements_week2.txt records dependencies. src/build_week2_report.py renders this report from computed measurements; it is not part of scoring.

3. Reference conditions

Lexical matching alone is the primary baseline. Semantic matching alone is an additional comparison, allowing assessment of whether the hybrid adds value over either component. All three methods use identical jobs, candidate resumes, labels and tie-breaking rules. A perfect ranking has NDCG@3 = 1.0; zero relevant candidates in the first three gives Precision@3 = 0.

4. Frozen inputs and experimental procedure

The inputs contain 3 fictional jobs, 18 fictional resumes and all 54 job-resume pairs. The 9 Week 1 resumes and their 27 labels are preserved, with 9 new resumes and 27 labels added before scoring. Each job family receives an additional direct match and partial match; three keyword-heavy negatives describe sales or recruiting roles that mention technical tools without performing the required tasks. Original paraphrased examples remain in the candidate pool.

The relevance rubric is 2 for substantial intended task/tool coverage, 1 for partial task coverage, and 0 for no demonstrated relevant task coverage within the constructed scenario. Labels are author-defined and not independent annotations. Added-case rationales are in data/week_2/added_scenarios.csv. SHA-256 digests in manifest.json freeze all input CSV files. The experiment aborts if their bytes change.

Before execution, the protocol fixed K = 3, relevance threshold >= 1, exponential gain 2^grade - 1, lexical hybrid weight 0.5, and resume_id ascending lexicographically for tied scores. The pretrained export revision is 751bff37182d3f1213fa05d7196b954e230abad9. Inference uses CPUExecutionProvider with one intra-op and one inter-op thread. No randomized sampling is used.

For each job, rank all 18 resumes independently under each method. Precision@3 is the number of grade-1 or grade-2 resumes in the first three divided by 3. NDCG@3 divides the discounted gain of the first three by the maximum possible gain for that same job and candidate pool; discounts are log2(rank + 1). Report per-job measurements and unweighted macro averages across the three jobs.

There is no supervised train/test split because this experiment neither trains nor tunes a model. TF-IDF IDF statistics use the fixed candidate collection, a collection-dependent retrieval setting. The constructed benchmark is a development benchmark, not an independent held-out test. Future tuning and final evaluation require separate query sets and independent judgments.

5. Reproduce the experiment

From the repository root, create a Python 3.12 environment and install src/requirements_week2.txt. The first download requires internet and about 91 MB for model assets. Subsequent runs can use the cached assets offline. Model files are excluded from Git. The manifest records the fixed protocol and hashes of the committed inputs. To reconstruct the benchmark in a separate directory, run python src/export_week2_benchmark.py path/to/new-directory.

```
python -m pip install -r src/requirements_week2.txt
python src/build_week2_report.py
```

After installing dependencies, open src/week1_resume_job_foundation.ipynb from the repository root or src directory, restart the Python kernel and run all cells from the beginning. The original Week 1 cells run first. The appended Week 2 cells check the committed benchmark against its manifest, verify the pinned model, execute all ten tests, and generate the experiment outputs. The report-rendering command is optional and runs after the notebook.

Results are saved under results/week_2/. Re-running overwrites generated outputs. execution_summary.json records exact runtime versions, protocol, input and model hashes, truncation count, and descriptive execution times. Timing is environment-dependent and is not used as a comparative research metric. reportlab is needed only for PDF generation; Jupyter is optional for the notebook.

6. Initial experimental results

Method	Macro Precision@3	Macro NDCG@3
Hybrid	0.666667	0.744880
Lexical	0.666667	0.724399
Semantic	0.777778	0.823093

Hybrid minus lexical macro NDCG@3 is +0.020481, while macro Precision@3 is unchanged. The NDCG change comes entirely from J2 (web development), where the grade-0 recruiting example moves from rank 2 to rank 3. J1 and J3 show no top-three metric improvement. Thus, the hybrid changes ordering quality on one job without retrieving more relevant resumes in the first three.

Semantic-only scoring has the highest averages. For J2 it ranks R4, R13 and the paraphrased R5 first, second and third, giving Precision@3 = 1 and NDCG@3 = 1. The hybrid instead retains grade-0 R15 at rank 3. This is evidence against assuming that a fixed blend must outperform the semantic component alone.

The keyword-negative data-analysis resume R12 remains rank 2 for all three methods. The IT-support sales resume R18 remains rank 3 for all three. These failures show that semantic similarity can still reward topical overlap despite explicit statements of missing task experience. The comparison retained the original labels and weights.

rankings.csv records every score and position. metrics_by_job.csv contains the nine method-job measurements. metrics_summary.csv contains the macro means. paired_comparison.csv records per-job differences, and top3_cases.csv identifies the examples supporting this interpretation. These are computed outputs, not predetermined demonstration values.

7. Validation performed

Ten automated tests passed: known Precision@3, perfect and zero NDCG, hand-calculated NDCG, agreement with scikit-learn NDCG using exponential gains, deterministic tie-breaking, attention-mask pooling, hybrid endpoint identities, unknown-vocabulary behavior, and invalid-K rejection. These tests check mathematical and numerical behavior; they do not validate the relevance judgments.

The experiment also checks unique identifiers, nonmissing and nonempty texts, complete and unique job-resume pairs, valid relevance grades, and unchanged input digests. It requires finite bounded scores, 384-dimensional embeddings and unit-length normalized vectors. No input text reached the configured truncation length. Scoring functions do not receive relevance labels; labels are joined only for evaluation.

8. Problems, limitations and next work

This is a very small, intentionally constructed benchmark with three queries. Near-duplicate direct matches and family-based labels make it easier than realistic resume screening. The added keyword negatives are diagnostic cases, not a representative sample of applicants. Author-defined labels may reflect design assumptions, and topical similarity is not evidence of qualification, employment suitability or hiring performance.

The observed improvement is one rank change on one job. No significance test or generalization claim is justified by this query count. Precision@3 ignores differences between grades 1 and 2, while NDCG depends on the chosen gain function. Macro averages can hide failures, so per-job measurements and candidate-level outputs are retained.

The equal-weight blend combines scores with different distributions without calibration. This is a predeclared reference rule rather than an optimized method. MiniLM is a general-purpose similarity model and does not reliably reason about negation, missing credentials or job requirements. This milestone therefore supports the feasibility of comparison, with mixed preliminary evidence for the hypothesis.

Next priorities: (1) obtain a larger, diverse set of queries and independently judged pairs with documented permission; (2) separate development queries from a frozen final evaluation set before any weight selection; (3) analyze paraphrases, partial qualifications and negation by case type with adequate counts; (4) compare predeclared alternative weights only on development data and evaluate once on held-out queries; (5) assess uncertainty at the query level after increasing the query count. These are Week 3 plans, not completed experiments.

Environment and provenance

Executed versions: python 3.12.14, numpy 2.3.5, pandas 2.2.3, scikit-learn 1.8.0, onnxruntime 1.31.1, tokenizers 0.23.3. Exact model and tokenizer digests are recorded in results/week_2/execution_summary.json. ONNX is used to run the pretrained sentence embedding model without the larger PyTorch runtime; it does not introduce a different semantic objective.

Method references

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP-IJCNLP. https://aclanthology.org/D19-1410/

Sentence Transformers. all-MiniLM-L6-v2 model card (model architecture, pooling and 256-token truncation). https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Xenova. all-MiniLM-L6-v2 ONNX export, revision 751bff37182d3f1213fa05d7196b954e230abad9. https://huggingface.co/Xenova/all-MiniLM-L6-v2

scikit-learn. ndcg_score and TfidfVectorizer API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ndcg_score.html ; https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html